

SUMMARY

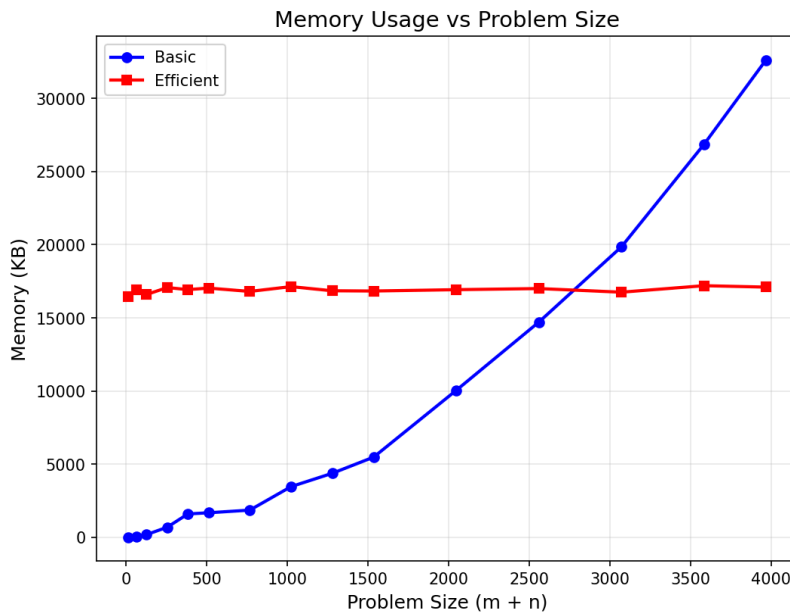
USC ID/s: 6576722945, 8349439767, 5561147067, 5497127370

Datapoints

M+N	Time in MS (Basic)	Time in MS (Efficient)	Memory in KB (Basic)	Memory in KB (Efficient)
16	0.0379	0.0639	16.0000	16432.0000
64	0.3409	0.6032	56.0000	16940.0000
128	1.2450	2.0921	188.0000	16600.0000
256	5.0511	8.8260	692.0000	17072.0000
384	10.8159	18.6641	1600.0000	16932.0000
512	19.7630	33.5071	1680.0000	17036.0000
768	53.1039	78.2363	1860.0000	16808.0000
1024	109.3073	144.1841	3468.0000	17136.0000
1280	158.4880	226.4540	4380.0000	16852.0000
1536	279.5680	320.9143	5484.0000	16836.0000
2048	384.1949	602.2298	10048.0000	16928.0000
2560	617.5113	954.6912	14712.0000	17004.0000
3072	885.3590	1349.8719	19856.0000	16756.0000
3584	1294.3759	1863.2872	26860.0000	17200.0000
3968	1528.9221	2382.3297	32624.0000	17104.0000

Insights

Graph1 – Memory vs Problem Size (M+N)



Nature of the Graph (Logarithmic/ Linear/ Polynomial/ Exponential)

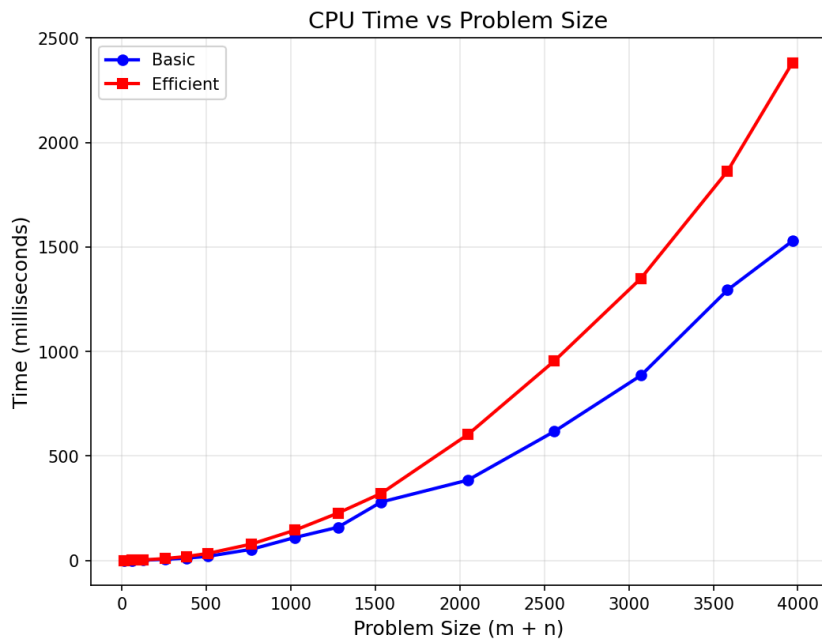
Basic: Polynomial

Efficient: Linear (Constant)

Explanation:

The memory consumption for the basic algorithm grows as the problem size (m+n) increases. This is because it stores the entire dynamic programming table of size (m+1) x (n+1) resulting in a space complexity of $O(mn)$. As shown in the table and graph memory scales from around 16 KB to 32 MB. For the efficient algorithm, the memory profile remains flat, around 16MB, due to the fact that it only stores two rows of the Dynamic programming table at any time reducing the space complexity to $O(\min(m, n))$. The high baseline (~ 16 MB) can be attributed to the Python interpreter and imported libraries (psutil) and to a lesser extent the operating system and current system config at the time the program is run.

Graph2 – Time vs Problem Size (M+N)



Nature of the Graph (Logarithmic/ Linear/ Polynomial/ Exponential)

Basic: Polynomial

Efficient: Polynomial

Explanation:

Both the basic and efficient algorithms have a similar quadratic growth trend. As the problem size (m+n) increases, the runtime for both algorithms increases. There is a gap, the efficient algorithm is slower than the basic algorithm, but this is expected as the memory efficient divide and conquer strategy requires re-computing optimal split points at each recursive stem. It also performs a forward and backward pass for each division, leading to twice as many total operations as the basic approach (single pass).

Contribution

Equal contribution